Scheduled-Work Queue Skill

Purpose

Teaches an agent to use the **scheduled-work** MCP server to record background / deferred work and — crucially — to **verify uncertain, blocked, or overdue work before reporting it done**. This is the teeth behind the `REMINDER` / `BACKGROUND` action-prefix (§11.4.140): when a REMINDER fires, the agent queries the queue for work whose real outcome is not yet confirmed and re-checks it, instead of guessing that it finished (§11.4.6 no-guessing, §11.4.108 verify-before-report).

When to use

- The operator dispatches background/deferred work ("do X later", "remind me to Y").
- A REMINDER arrives and the agent must decide what still needs confirming.
- Before an autonomous loop declares its queue empty (§11.4.87 / §11.4.126) — first confirm no item is still `blocked` / `uncertain` / `overdue`.

MCP tools (server name: `scheduled-work`)

- `create_work_item {title, description?, status?, due_at?, tags?, source?, notes?}` — record a work item. `status` ∈ `scheduled|in-progress|blocked|uncertain|done|cancelled` (default `scheduled`); `due_at` is RFC3339.
- `list_work_items {status?}` — list items (optional effective-status filter, incl. derived `overdue`).
- `get_work_item {id}` — one item by id.
- `update_work_item_status {id, status, notes?}` — move an item along.

- `mark_work_item_done {id, notes?}` — close it. **Only call AFTER you have verified the real outcome** — put the captured evidence in `notes`.
- `list_overdue_work {}` — open items past their due time.
- `list_needs_verification {}` — open items (`blocked` / `uncertain` / `overdue`) a REMINDER MUST confirm before reporting done.

Recommended flow

1. **Record**: on dispatching background work, `create_work_item` with a clear title + `status:"in-progress"` (or `blocked` / `uncertain` if the outcome is not yet known).
2. **On REMINDER**: call `list_needs_verification`. For each returned item, re-check the real system state (run the probe / read the sink / re-run the test), gather captured evidence.
3. **Confirm-before-done**: only when the evidence shows the work genuinely succeeded, `mark_work_item_done` with the evidence path in `notes`. If it did NOT succeed, `update_work_item_status` back to `in-progress` / `blocked` and keep working (§11.4.94 zero-idle).
4. **Never** mark done on assumption — an item left `uncertain` is the honest state (§11.4.6).

Setup

The MCP server is the `scheduled-work-engine` Go binary. Build it once:

```bash
plugins/scheduled-work/build.sh   # or: cd scripts/scheduled-work-engine && go build -o bin/scheduled-work ./cmd/scheduled-work
```

Registration: `mcp/scheduled-work-mcp.json` (project-scoped) or the Claude Code plugin `plugins/scheduled-work/`.

Cross-references

- constitution/Constitution.md §11.4.6 (no-guessing)

- constitution/Constitution.md §11.4.108 (verify-before-report / runtime-signature)
- constitution/Constitution.md §11.4.140 (action-prefix — REMINDER/ BACKGROUND)
- constitution/Constitution.md §11.4.28 (decoupled, project-agnostic submodule)
- constitution/Constitution.md §11.4.116 (agent sync channel — sibling discipline)
- scripts/scheduled-work-engine/README.md (full engine docs + REST/MCP/ ACP surfaces)